

```

'sequence': 'this is a test.',
'token': 3231,
'token_str': 'test'},
{'score': 0.025175178423523903,
'sequence': 'this is a game.',
'token': 2208,
'token_str': 'game'},
{'score': 0.024909017607569695,
'sequence': 'this is a lie.',
'token': 4682,
'token_str': 'lie'}}

from transformers import pipeline
unmasker = pipeline('fill-mask', model='bert-base-uncased')
myprediction ("He is a [MASK].")

```

The output is:

```

[{'score': 0.17371997237205505,
'sequence': 'he is a christian.',
'token': 3017,
'token_str': 'christian'},
{'score': 0.08878538012504578,
'sequence': 'he is a democrat.',
'token': 7672,
'token_str': 'democrat'},
{'score': 0.06659623980522156,
'sequence': 'he is a republican.',
'token': 3951,
'token_str': 'republican'},
{'score': 0.03911091387271881,
'sequence': 'he is a vegetarian.',
'token': 23566,
'token_str': 'vegetarian'},
{'score': 0.036758508533239365,
'sequence': 'he is a catholic.',
'token': 3234,
'token_str': 'catholic'}]

```

Prediction of a word using a pre-trained NLP model

We can predict a word using a pre-trained NLP model as follows:

```

import torch
from pytorch_transformers import BertTokenizer, BertModel,
BertForMaskedLM
# Loading of Tokenizer
mytokenizer = BertTokenizer.from_pretrained('bert-base-
uncased')
# Input Tokenization
text = "[CLS] Who was Puppet Expert ? [SEP] Puppet Expert was
a puppeteer [SEP]"
t_Text = mytokenizer.tokenize(text)

```

```

# Masking of Token
masked_index = 8
t_Text[masked_index] = '[MASK]'
assert t_Text == ['[CLS]', 'who', 'was', 'Puppet', 'Expert',
'?', '[SEP]', 'Puppet', '[MASK]', 'was', 'a', 'puppet',
'##eer', '[SEP]']
# Conversion of Token
i_token = tokenizer.convert_tokens_to_ids(t_Text)
segments_ids = [0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1]
# Conversion of Inputs to Tensors
t_tensor = torch.tensor([i_token])
segments_tensors = torch.tensor([segments_ids])
# Loading of Weights from Pre-Training Model
model = BertForMaskedLM.from_pretrained('bert-base-uncased')
model.eval()
# Invocation of GPU
t_tensor = t_tensor.to('cuda')
s_tensors = segments_tensors.to('cuda')
model.to('cuda')

# Prediction of tokens
with torch.no_grad():
    outputs = model(t_tensor, token_type_ids=s)
    predictions = outputs[0]
pred_index = torch.argmax(predictions[0, masked_index]).
item()
pred_token = tokenizer.convert_ids_to_tokens([pred_index])[0]
assert pred_token == 'Expert'
print('Pred token is:', pred_token)

```

The output is:

```

100%|██████████| 231508/231508 [00:00<00:00, 703114.82B/s]
100%|██████████| 433/433 [00:00<00:00, 81193.38B/s]
100%|██████████| 440473133/440473133 [00:20<00:00,
21132461.45B/s]
Predicted token is: Expert

```

Research scholars, academicians and practitioners working in the domain of speech and natural language processing can use free and open source pre-trained models for their research work as these enable a high degree of accuracy and performance with real-world data sets. The models available in such cloud based platforms are quite effective for dynamic applications including audio forensics, speech recognition, speech-to-text translation, language analytics, and many others. 

 By: Dr Gaurav Kumar

The author is associated with various academic and research institutes, where he delivers expert lectures and conducts technical workshops on the latest technologies and tools.